

Understanding Guide — the explain-it-to-me-from-scratch version

Zomato Delivery · North vs South Regional Classification · Team Hmmmmmmmmmm

How to read this document. Each concept is presented three ways: (1) plain English — what it is in one sentence, no jargon; (2) why we care — what it actually measures; (3) our value — the specific number in our project, and what to say if a judge asks. Read top to bottom; nothing is skipped or assumed.

0. The three numbers you must say correctly

If you remember nothing else, remember these three AUC values and what they mean:

Number	Plain English	Our value	What to say
GEO AUC	Using only the latitude & longitude, can we classify each restaurant as South or North?	1.000	Perfect, because the North is literally defined by location. T
BEHAVIOUR AUC	Without using location, just using rider/orders/restaurant features, can we classify each restaurant as South or North?	0.508	Essentially random (0.50 is from flipping a coin). Means the way delivery
STRUCTURAL AUC	Behaviour + one extra feature: the distance between restaurant and location of rider's orders	0.914	Staps and Geometry for layout of cities differs — South c

1. What is AUC? (the most important metric to explain)

1.1 Plain English

AUC stands for **Area Under the (ROC) Curve**. Imagine you pick two orders at random — one truly from the South, one truly from the North. You ask the model to score each one with a probability of being South. AUC is the probability that the model gives the truly-South order a *higher* score than the truly-North order.

- AUC = **1.0** → the model always ranks South higher than North. Perfect separation.
- AUC = **0.5** → the model is guessing. South and North get the same score on average. Coin flip.
- AUC = **0.0** → the model is perfectly wrong. Almost never happens; if it does, you've flipped the labels.
- AUC = **0.7-0.8** → useful predictor. AUC > **0.9** = strong. AUC < **0.55** = essentially random.

1.2 Why we use AUC instead of accuracy

Accuracy = (# of correct predictions) / (total predictions). It depends on where you put the decision threshold (by default 0.5 — 'if probability of South > 0.5, predict South'). Accuracy is misleading when classes are imbalanced. In our data, North is 61% of rows — a model that always predicts 'North' gets 61% accuracy without learning anything. AUC sidesteps this by looking at *ranking* across all possible thresholds.

1.3 What does GEO AUC = 1.000 mean specifically?

It means that on every single one of the 41,944 orders, when we trained the model using only the latitude and longitude features, every truly-South order received a higher predicted probability than every truly-North order. The model got it right 100% of the time. **This is expected and not impressive** — we defined South as 'state belongs to {Kerala, TN, AP, Telangana, Karnataka}', which is itself derived from latitude/longitude (state borders are geographic). The GEO model is a **sanity check** that confirms our label vector is correctly aligned with coordinates. If GEO AUC were 0.8 instead of 1.0, it would mean we had labelled rows incorrectly. Because it's 1.0, we know our 22-city → region map is bug-free.

If a judge asks 'why is your AUC 1.0, isn't that overfitting?' say: 'AUC = 1.0 here proves the labels are clean. Region was defined by state, which is determined by coordinates. So a model trained on coordinates can recover region perfectly. This isn't the model being good — it's the data being internally consistent. The interesting models are the next two.'

1.4 AUC = 0.508 means almost-random — why is that the interesting result?

0.508 is within Monte Carlo noise of 0.500. The behaviour-only model (no location, no distance, no time-leak features) is unable to distinguish South from North orders better than guessing. This is the **NEGATIVE** finding that, after six independent stress tests, becomes the headline: **Zomato's operational behaviour is statistically identical North vs South in this dataset**. It is not bad performance; it is honest reporting of the absence of a behavioural signal.

2. Other metrics we report (and what each tells you)

2.1 F1 score

Combines precision (of orders we PREDICTED South, how many were actually South?) and recall (of orders that ARE truly South, how many did we catch?) into a single number, range 0 to 1.

```
Precision = TP / (TP + FP)
Recall = TP / (TP + FN)
F1 = 2 · Precision · Recall / (Precision + Recall)
```

Our structural model has F1 = 0.952. That means precision and recall are both high and balanced — when the model says 'South', it's right ~95% of the time, and it catches ~95% of the actual South orders.

2.2 Balanced accuracy

Average of recall on the South class and recall on the North class. Unlike plain accuracy, it doesn't favour the majority class. Our value: **0.964**. A balanced-accuracy of 0.5 would be random; 1.0 would be perfect.

2.3 Confusion matrix

A 2×2 table that counts every prediction:

	Model predicted NORTH	Model predicted SOUTH
Truly NORTH	TN (true negative)	FP (false positive)
Truly SOUTH	FN (false negative)	TP (true positive)

TP, FP, FN, TN are simply how many rows fell into each cell. Precision, recall, accuracy, F1 are all computed from these four numbers.

3. The three models — what each one is and why

3.1 Model A — GEO (the calibration model)

What it does: takes five features — Restaurant_latitude, Restaurant_longitude, Delivery_location_latitude, Delivery_location_longitude, and haversine distance_km — and outputs the probability that the order is from South India.

Our result: 5-fold AUC = 1.000, F1 = 1.000, balanced accuracy = 1.000. All five folds got AUC 1.0000 exactly.

What it tells us: the label vector is clean. The 22-city → region map we built has no errors. This isn't a real predictive achievement — it's a sanity check. **Why do we report it?** If we only showed Models B and C, judges could ask 'how do you know your labels are right?'. Model A is the audit trail that proves they are.

3.2 Model B' — BEHAVIOUR-only (the honest baseline)

What it does: uses 12 numeric + 6 categorical behaviour features — rider age, rider rating, prep time, order hour, peak-hour flags, weekend flag, vehicle condition, multi-delivery count, weather conditions, traffic density, vehicle type, order type, city tier, festival. **No coordinates. No distance. No month or day_of_week** (those leak the data-collection window — see section 5).

Our result: 5-fold AUC = 0.508, F1 = 0.433. Fold-by-fold: 0.514, 0.515, 0.500, 0.500, 0.508. Two folds are exactly 0.500 — the model literally is guessing.

What it tells us: none of the behavioural features carry regional signal. Zomato's deliveries unfold the same way whether they're in Bangalore or Bhopal.

File on disk: models/region_classifier_v1.txt (951 KB, ~330 trees).

3.3 Model C — STRUCTURAL (the headline model)

What it does: Model B' *plus* one feature — distance_km, the haversine distance between the restaurant pickup and the customer drop, for each individual order.

Our result: 5-fold AUC = 0.996, F1 = 0.952. GroupKFold AUC (when held-out cities differ from training cities) = 0.911. The fold-4 number is 0.642 (anomaly explained in section 5.3).

What it tells us: Even though pickup-to-drop distance has nearly identical medians in both regions (South 9.2 km, North 9.0 km), the *distributions differ in shape* (KS $p \approx 10^{-10}$). South Indian cities have a structurally different layout of where restaurants are placed relative to where customers live. Cities like Bangalore or Chennai have a different fingerprint than Mumbai or Jaipur — and this geometric difference is what the model picks up on.

File on disk: models/region_classifier_v2_structural.txt (1.9 MB, ~276 trees).

How can distance_km be 'structural' and not 'geographic'?

Geographic features tell you WHERE you are (latitude 12.97°, longitude 77.59° = Bangalore). Structural features tell you about the SHAPE of the operation. distance_km is the gap between pickup and drop for a single order — two different cities can produce the same distance_km value. We never tell the model that 12.97°/77.59° is Bangalore; we just give it the distance number, and the model learns 'orders in South tend to span 9–14 km with these particular tail behaviours, orders in North span 8–12 km with different tails'.

4. The statistical tests we used (and their results)

4.1 Mann-Whitney U test

Plain English: Are these two groups of numbers similar?

How: Rank everyone from all groups combined (smallest to largest). Sum the ranks for each group. If the groups are similar, the rank sums should be roughly equal. If one group is consistently higher or lower, the rank sums differ — and the U statistic captures that.

Why we use it: our data is not Gaussian, so a t-test would be unsafe. Mann-Whitney makes no distributional assumption.

What p-value means: if there were truly no difference between South and North on this column, the probability of seeing a U statistic at least as extreme as ours is p. Small p (< 0.05) → reject 'no difference'.

4.2 Chi-square (χ^2) test of independence

Plain English: Do two categorical variables go together or are they unrelated?

How: Build a table of counts (rows = traffic categories, columns = region). Compute what the table would look like if region and traffic were independent — that's the 'expected' table. Compare to what we actually see. The χ^2 statistic measures total squared difference, normalised by expected counts.

Why we use it: weather, traffic, vehicle, order type are categorical; we want to know if their distribution differs by region.

4.3 Kolmogorov-Smirnov (KS) two-sample test

Plain English: Are two whole distributions identical, not just their medians?

How: Plot the cumulative distribution function (CDF) of each group. The KS statistic D is the maximum vertical distance between the two curves at any point.

Why we use it: Mann-Whitney can miss differences in the tails or shape. Two distributions can have identical medians but completely different spreads or skews — KS catches that.

Our key finding: distance_km has KS D = 0.088, $p \approx 6.7 \times 10^{-11}$ — distributions differ in shape even though medians match (South 9.2 vs North 9.0). This is what justifies Model C.

4.4 All eleven univariate tests at a glance

#	Feature	Test	Statistic	p-value	Verdict
01_target	Delivery time differs by region	Mann-Whitney U	U=207941110	2.13e-01	North > South
02_weather	Weather distribution differs	Chi-square	chi2=7.7	2.64e-01	Same
03_traffic	Traffic density differs	Chi-square	chi2=1.0	7.95e-01	Same
04_multi	Multi-delivery norms differ	Mann-Whitney U	U=199693348	3.90e-01	North > South
05_age	Rider age skew	Mann-Whitney U	U=192135843	6.51e-01	North > South
06_rating	Rider rating skew	Mann-Whitney U	U=191711132	9.11e-01	North > South
07_vehicle	Vehicle preferences differ	Chi-square	chi2=3.9	2.76e-01	Same
08_order_type	Order-type mix differs	Chi-square	chi2=0.7	8.78e-01	Same
09_festival	Festival incidence differs	Chi-square	chi2=2.3	1.32e-01	Same
10_hour	Time-of-day rhythm	KS 2-sample	D=0.006	8.30e-01	Same
11_city_tier	City-tier mix differs	Chi-square	chi2=3.0	2.29e-01	Same

All eleven p-values are > 0.05. By the standard significance threshold, none of these univariate differences are statistically real. This is the 'apparent flat finding' that motivated the deeper checks.

5. The six stress tests — trying to break the finding

After eleven univariate tests all said 'no difference', and the behaviour-only model AUC was 0.508 (random), we ran six MORE checks in case we were missing something. Here's each one in plain English.

5.1 KS distribution-shape on every numeric column

The worry it addresses: 'maybe the medians match but the tails differ — Mann-Whitney would miss that'.

What we did: ran KS test on every numeric column (delivery time, distance, age, rating, etc.). **Result:** only distance_km significant ($D=0.088$, $p \approx 10^{-10}$). Every other column had near-identical shapes.

5.2 Dispersion ratio (std comparison)

The worry: 'maybe both regions have the same average but South has more variability'. **What we did:** computed south_std / north_std for every numeric column. **Result:** all ratios between 0.98 and 1.04. Variability is the same in both regions.

5.3 Interaction features ablation

The worry: 'two features individually might not differ but their interaction could'. **What we did:** engineered 5 interaction features (traffic × hour, weather × traffic, vehicle × multi, distance × age, rating × traffic). Added them one at a time on top of behaviour-only and watched the AUC. **Result:** 4 out of 5 added zero lift. The one that lifted (distance_per_age) was distance_km in disguise.

5.4 GroupKFold by city

The worry: 'maybe the model memorises per-city patterns and looks better than it really is'. **What we did:** in normal KFold we hold out random ROWS; in GroupKFold we hold out entire CITIES. So the training cities and test cities don't overlap. **Result:** behaviour-only stayed at AUC 0.506 (random). Structural reached 0.911 on held-out cities, with one fold (Fold 4, held Kochi + Hyderabad) dropping to 0.642 — a real limit (see section 7).

5.5 Per-South-state internal classifier

The worry: 'is South India internally uniform, or are Kerala, TN, AP, Karnataka, Telangana all different from each other?'. **What we did:** trained a 4-way classifier on the South subset only, predicting state. **Result:** accuracy = 0.377 vs random 0.250. South India is weakly internally heterogeneous — enough to explain the Fold-4 dip but not enough to overturn the 'behaviourally uniform' headline.

5.6 CatBoost cross-check

The worry: 'is the 0.508 AUC a LightGBM-specific artefact?'. **What we did:** retrained the exact same task with CatBoost, a different gradient-boosting library that handles categoricals differently. **Result:** CatBoost AUC = 0.507, within 0.001 of LightGBM. Not a framework artefact.

6. The city-code sanity check (Goa was 'the worst offender')

We labelled each row by extracting the city prefix from `Delivery_person_ID` (e.g. `KOCRES16DEL01` → `KOC` → Kochi → Kerala → South). For this to be trustworthy, we needed to verify that the labels actually match reality.

6.1 What we did

For each of the 22 city codes, we already knew the expected lat/lon centroid (e.g. Bangalore $\approx 12.97^\circ\text{N}$, 77.59°E). We computed the median latitude and median longitude of every row carrying that city code, then measured the distance (in kilometres) between the measured median and the expected centroid.

6.2 Why a 'worst offender'?

If all 22 cities had medians within (say) 10 km of their expected centroids, the labelling is rock-solid. If one city's median was off by 500 km, that would suggest the rider-ID prefix is mislabelled or the coordinate column is corrupted for those rows. We sort the table by `km_off` and report the worst one — to be transparent that we checked.

6.3 The result

All 22 cities passed. The worst offender is **Goa (GOA)** at **44.9 km** off centroid: our orders' median was lat 15.517, lon 73.767, expected 15.3, 74.12. 45 km is well under our 250 km tolerance — likely just that the GOA centroid we used was Panaji (the capital), while many orders come from elsewhere on the Goan coast (Margao, Vasco). **19 of the 22 cities are within 7 km** of their expected centroids — verification is exceptionally clean.

If asked 'what does worst offender mean?' say: 'It's the city whose actual median lat/lon was farthest from the expected centroid in our city-code map. Goa was 45 km off, well within our 250 km tolerance. It just means the labelled centroid for Goa state was Panaji, but Goa's orders are spread across Panaji, Margao, and Vasco. No correction needed — the city code still correctly identifies the region.'

7. Every graph in our deliverables — axes explained

[fig_reg_auc_compare.png](#)

What it is: Bar chart with 5 bars for the GEO model and 5 bars for the BEHAVIOUR model.

X axis: Fold number (1, 2, 3, 4, 5)

Y axis: ROC-AUC (0.5 = random, 1.0 = perfect)

How to read: Grey bars all sit at 1.0 (geo perfect). Teal bars all sit near 0.5 (behaviour random). The visual gap is the headline finding.

[fig_reg_shap_bar.png](#)

What it is: Horizontal bars showing the average importance of each behaviour feature in Model B.

X axis: Mean absolute SHAP value (how much the feature shifts the prediction)

Y axis: Feature name

How to read: Month dominates (SHAP $\approx$ 0.45); every other feature has SHAP $<$ 0.05. This is what revealed the temporal leak.

[fig_reg_shap_beeswarm.png](#)

What it is: Dotted strip plot: each dot is one row from a 1,500-row sample, plotted along the SHAP-value axis per feature.

X axis: SHAP value (negative = pushes prediction toward North, positive = toward South)

Y axis: Feature name (ranked top to bottom by importance)

How to read: Colour of each dot = the feature's actual value (red=high, blue=low). Lets you see how a feature's value relates to its directional impact.

[fig_reg_structural_shap.png](#)

What it is: Same as fig_reg_shap_bar but for the structural model (Model C, with distance_km added).

X axis: Mean absolute SHAP value

Y axis: Feature name

How to read: distance_km dominates with SHAP $\approx$ 4.87. Every other feature is below 0.15. This is the visual proof that distance_km is doing all the work in Model C.

[fig_reg_deep_compare.png](#)

What it is: Six bars summarising all our model variants under GroupKFold-by-city.

X axis: Model variant

Y axis: ROC-AUC

How to read: Geo perfect (1.0), all behaviour variants near 0.5 (random), structural with distance jumps to $\sim$ 0.87. The dotted line at 0.5 is random; values above it carry signal.

[fig_reg_confusion.png](#)

What it is: 2x2 heatmap of the out-of-fold predictions vs truth at threshold 0.5.

X axis: Predicted label (North on the left column, South on the right)

Y axis: True label (North on top row, South on bottom row)

How to read: Top-left and bottom-right cells = correct predictions (TN, TP). Top-right and bottom-left = mistakes (FP, FN). For the structural model, almost everything is on the diagonal.

fig_reg_curves.png

What it is: Three-panel diagnostic plot for Model C.

X axis: Panel 1 (ROC) — False Positive Rate; Panel 2 (PR) — Recall; Panel 3 (Calibration) — Mean predicted probability

Y axis: Panel 1 — True Positive Rate; Panel 2 — Precision; Panel 3 — Observed fraction South

How to read: All three curves should hug the top/left ideal. Calibration curve being close to the dashed $y=x$ line means the predicted probability is close to the actual probability.

fig_reg_01_target.png ... fig_reg_11_citytier.png

What it is: Eleven side-by-side comparisons (violin or stacked-bar) of behaviour features in North vs South.

X axis: Region (North on left, South on right)

Y axis: The feature value (delivery time, age, rating, etc.) OR the % share within each region (for categoricals)

How to read: Visually, North and South distributions look almost identical in every one of these plots — confirming the 'no behavioural signal' finding visually.

8. Jargon cheat sheet (defend in one sentence)

Term	One-sentence definition
AUC	Probability the model ranks a random South higher than a random North. 0.5=random, 1.0=perfect.
F1 score	Harmonic mean of precision and recall, range 0 to 1, higher is better.
Precision	Of the orders we predicted South, what fraction was actually South.
Recall	Of orders that were actually South, what fraction did we correctly catch.
Balanced accuracy	Average of recall on each class — not skewed by class imbalance like raw accuracy.
Confusion matrix	2×2 table of true vs predicted counts (TN, FP, FN, TP).
KFold	Split rows into K equal folds; train on K-1, predict on 1, rotate, average.
StratifiedKFold	KFold that preserves the South/North ratio in every fold.
GroupKFold	KFold variant where rows from the same city never appear in both train and test.
OOF (out-of-fold)	For each row, the prediction made when that row was in the validation fold.
Mann-Whitney U	Non-parametric test for whether two samples have the same median.
Chi-square	Test for independence between two categorical variables.
KS test	Test for whether two whole distributions are identical (not just medians).
Spearman p	Correlation on ranks; measures monotonic (not necessarily linear) relationship.
Haversine	Great-circle distance between two lat/lon points on a sphere.
LightGBM	Gradient-boosted decision tree library, leaf-wise growth, handles categoricals natively.
CatBoost	Another gradient-boosting library used for cross-checking.
Class weight=balanced	Re-scales loss inversely to class frequency so the minority class isn't ignored.
Early stopping	Stop adding trees once validation AUC plateaus, to prevent overfit.
SHAP (TreeExplainer)	Per-feature attribution telling you how much each feature shifted the prediction.
Data leak / temporal leak	When a feature carries information not available at prediction time, or is a proxy for the label.
Ablation	Adding or removing one feature at a time to measure its individual contribution.
Centroid	The geographic centre point of a city, used as the expected lat/lon for that city's orders.

Pre-pitch ritual. Read section 0 (the three numbers) one more time. Open the deck, walk through slides 1, 5, 8, 10 in your head. Open a terminal in the repo folder and have `python predict.py --input "data/raw/Zomato Dataset.csv" --output preds.csv` ready to paste if asked for a demo. Take this PDF with you.

Repo: github.com/TheClazer/Zomato-delivery-estimation · **Generator:** `notebooks/_build_understanding_guide.py` from `live_regional_findings.json`